ОТЧЁТ

по лабораторной работе №5

«Функциональные возможности Python: функции высшего порядка, итераторы и генераторы»

Вариант 20. Грузы

1. Цель работы

Получить практические навыки обработки коллекций средствами функционального стиля Python: функции высшего порядка, lambda, map, filter, reduce, sorted с ключом, any и all, генераторы списков и словарей, yield и собственный итератор.

2. Постановка индивидуального задания

Вариант 20. Грузы. Для груза хранятся номер, отправитель, получатель, масса, стоимость доставки и статус. Нужно отобрать доставленные грузы, получить номера, отсортировать по массе, посчитать общую стоимость, проверить наличие грузов тяжелее заданной массы и выдать генератором грузы выбранного статуса.

Есть базовый класс Cargo и два наследника: обычный и срочный груз. Обработка вынесена в operations.py, генератор и итератор — в generators.py.

3. Краткие теоретические сведения

Функция в Python является объектом: её можно передать в другую функцию. Функция высшего порядка принимает функцию как аргумент или возвращает функцию. lambda задаёт короткое безымянное выражение.

map применяет функцию к каждому элементу и возвращает итератор. filter оставляет элементы, для которых предикат истинен. reduce из functools сворачивает последовательность в одно значение. sorted(..., key=...) сортирует по результату ключа, не меняя исходный список.

any возвращает True, если хотя бы один элемент истинен, all — если истинны все. Генератор с yield выдаёт значения по одному и не держит всю последовательность в памяти. Итератор реализует __iter__ и __next__ и в конце бросает StopIteration.

4. Описание программы

Доставленные грузы отбираются через filter и lambda. Номера получаются через map. Общая стоимость считается reduce. Проверка тяжёлых грузов сделана через any, положительность цен — через all. Словарь «номер → стоимость» собран словарным включением.

Генератор cargos_by_status отдаёт подходящие грузы через yield. CargoIterator обходит только грузы не легче порога и завершается StopIteration. Отдельный пункт меню показывает, что список строится сразу, а генератор отдаёт элементы по next().

5. Исходный код программы

Ниже приведён исходный код разработанных файлов.

Листинг — main.py

```
"""
Лабораторная работа №5. Вариант 20.
Функциональная обработка грузов.
"""

import sys

from generators import CargoIterator, cargos_by_status
from models import ExpressCargo, StandardCargo
from operations import (
    all_have_cost,
    apply_to_each,
    costs_by_number,
    delivered,
    has_heavy,
    heavy_numbers,
    numbers_of,
    select_where,
    sort_by_mass,
    total_cost,
    unique_statuses,
)

if sys.platform == "win32":
    sys.stdout.reconfigure(encoding="utf-8")
    sys.stderr.reconfigure(encoding="utf-8")
    if not sys.stdin.isatty():
        sys.stdin.reconfigure(encoding="utf-8")



def read_line(prompt):
    if sys.stdin.isatty():
        return input(prompt).strip()
    raw = sys.stdin.readline()
    if raw == "":
        raise EOFError
    text = raw.rstrip("\r\n")
    print(f"{prompt}{text}")
    return text.strip()



def read_float(prompt):
    while True:
        text = read_line(prompt).replace(",", ".", 1)
        try:
            return float(text)
        except ValueError:
            print("Ошибка: введите число.")
```

```
def print_cargos(cargos, title):
    print(f"\n{title}:")
    if not cargos:
        print("Подходящих грузов нет.")
        return
    for cargo in cargos:
        print(" -", cargo.describe())


def sample_data():
    return [
        StandardCargo("G-101", "Озон", "Иванов", 12.5, 900, "доставлен"),
        ExpressCargo("G-102", "М.Видео", "Петрова", 4.2, 1500, "в пути", 6),
        StandardCargo("G-103", "Склад Юг", "Сидоров", 40.0, 2400, "принят"),
        ExpressCargo("G-104", "Озон", "Кузнецова", 2.1, 1100, "доставлен", 4),
        StandardCargo("G-105", "Ламода", "Орлов", 8.0, 700, "отменён"),
        ExpressCargo("G-106", "Склад Юг", "Иванов", 25.0, 3200, "доставлен", 12),
    ]


def show_lazy_difference(cargos):
    """Показывает, что генератор не строит список заранее."""
    print("\n----- Список и генератор -----")
    ready = [cargo.number for cargo in cargos if cargo.status == "доставлен"]
    print("Список номеров доставленных построен сразу:", ready)
    print("type(list) =", type(ready).__name__)

    lazy = cargos_by_status(cargos, "доставлен")
    print("Генератор создан, элементы ещё не вычислены. type =", type(lazy).__name__)
    print("Первый next():", next(lazy).number)
    print("Второй next():", next(lazy).number)
    print("Остаток генератора:", [cargo.number for cargo in lazy])


def print_menu():
    print()
    print("===== Обработка грузов =====")
    print("1. Показать все объекты")
    print("2. Отфильтровать доставленные")
    print("3. Получить номера")
    print("4. Отсортировать по массе")
    print("5. Общая стоимость доставки")
    print("6. Запустить генератор по статусу")
    print("7. Проверить наличие тяжёлых грузов")
    print("8. Собственный итератор")
    print("9. Сравнение списка и генератора")
    print("0. Выход")
    print()


def main():
    cargos = sample_data()
    print("Загружен учебный набор грузов:", len(cargos))
    print("Классы:", ", ".join(sorted({type(item).__name__ for item in cargos})))

    while True:
        print_menu()
        choice = read_line("Выберите действие: ")
        try:
            if choice == "1":
                print_cargos(cargos, "Все грузы")
            elif choice == "2":
                found = delivered(cargos)
                print_cargos(found, "Доставленные грузы (filter + lambda)")
            elif choice == "3":
                print("Номера (map):", ", ".join(numbers_of(cargos)))
                described = apply_to_each(cargos, lambda cargo: cargo.number)
                print("Та же выборка через функцию высшего порядка:", ", ".join(described))
            elif choice == "4":
                ordered = sort_by_mass(cargos)
                print_cargos(ordered, "По возрастанию массы (sorted + key)")
```

```
            elif choice == "5":
                print(f"Общая стоимость (reduce): {total_cost(cargos):.0f} руб.")
                print("Стоимости по номерам:", costs_by_number(cargos))
                print("Уникальные статусы:", ", ".join(sorted(unique_statuses(cargos))))
                print("У всех грузов стоимость > 0:", all_have_cost(cargos))
            elif choice == "6":
                status = read_line("Статус (принят / в пути / доставлен / отменён): ")
                print(f"Генератор выдаёт грузы со статусом «{status}» по одному:")
                count = 0
                for cargo in cargos_by_status(cargos, status):
                    count += 1
                    print(f"  yield → {cargo.describe()}")
                if count == 0:
                    print("  генератор не выдал ни одного значения")
            elif choice == "7":
                limit = read_float("Порог массы, кг: ")
                print(f"any(масса > {limit:.1f}): {has_heavy(cargos, limit)}")
                print("Номера не легче порога:", heavy_numbers(cargos, limit) or "нет")
                cheap = select_where(cargos, lambda cargo: cargo.cost < 1000)
                print_cargos(cheap, "Дополнительно: стоимость меньше 1000 (функция высшего порядка)")
            elif choice == "8":
                limit = read_float("Минимальная масса для итератора, кг: ")
                iterator = CargoIterator(cargos, limit)
                print("Обход собственным итератором:")
                try:
                    while True:
                        cargo = next(iterator)
                        print("  next →", cargo.describe())
                except StopIteration:
                    print("StopIteration: элементы закончились")
            elif choice == "9":
                show_lazy_difference(cargos)
            elif choice == "0":
                print("Работа программы завершена.")
                break
            else:
                print("Ошибка: выберите пункт меню от 0 до 9.")
        except EOFError:
            print("Ввод завершён.")
            break
        except ValueError as exc:
            print(f"Ошибка данных: {exc}")


if __name__ == "__main__":
    main()
```

Листинг — generators.py

```
"""Генератор грузов выбранного статуса и собственный итератор."""


def cargos_by_status(cargos, status):
    """Выдаёт грузы выбранного статуса по одному."""
    for cargo in cargos:
        if cargo.status == status:
            yield cargo


class CargoIterator:
    """Собственный итератор: обходит грузы тяжелее порога."""

    def __init__(self, cargos, min_mass):
        self._items = [cargo for cargo in cargos if cargo.mass >= min_mass]
        self._index = 0

    def __iter__(self):
        return self

    def __next__(self):
        if self._index >= len(self._items):
            raise StopIteration
        item = self._items[self._index]
        self._index += 1
```

```
        return item
```

Листинг — models.py

```
"""Классы грузов для функциональной обработки."""


class Cargo:
    """Базовый груз."""

    def __init__(self, number, sender, recipient, mass, cost, status):
        self.number = number
        self.sender = sender
        self.recipient = recipient
        self.mass = float(mass)
        self.cost = float(cost)
        self.status = status

    def kind(self):
        return "груз"

    def describe(self):
        return (
            f"{self.number} [{self.kind()}] {self.sender} → {self.recipient}, "
            f"{self.mass:.1f} кг, {self.cost:.0f} руб., {self.status}"
        )


class StandardCargo(Cargo):
    """Обычная доставка."""

    def kind(self):
        return "обычный"


class ExpressCargo(Cargo):
    """Срочная доставка: стоимость выше за счёт коэффициента."""

    def __init__(self, number, sender, recipient, mass, cost, status, hours):
        super().__init__(number, sender, recipient, mass, cost, status)
        self.hours = int(hours)

    def kind(self):
        return "срочный"

    def describe(self):
        return super().describe() + f", срок {self.hours} ч"
```

Листинг — operations.py

```
"""Функции высшего порядка и обработка коллекции грузов."""

from functools import reduce


def apply_to_each(items, operation):
    """Функция высшего порядка: применяет operation к каждому элементу."""
    return [operation(item) for item in items]


def select_where(items, predicate):
    """Функция высшего порядка: оставляет элементы, для которых predicate истинен."""
    return [item for item in items if predicate(item)]


def delivered(cargos):
    """Фильтрация доставленных грузов."""
    return list(filter(lambda cargo: cargo.status == "доставлен", cargos))
```

```
def numbers_of(cargos):
    """Получение номеров через map."""
    return list(map(lambda cargo: cargo.number, cargos))


def sort_by_mass(cargos, reverse=False):
    return sorted(cargos, key=lambda cargo: cargo.mass, reverse=reverse)


def total_cost(cargos):
    """Общая стоимость доставки через reduce."""
    if not cargos:
        return 0.0
    return reduce(lambda acc, cargo: acc + cargo.cost, cargos, 0.0)


def has_heavy(cargos, limit):
    """Есть ли грузы тяжелее заданной массы."""
    return any(cargo.mass > limit for cargo in cargos)


def all_have_cost(cargos):
    return all(cargo.cost > 0 for cargo in cargos)


def costs_by_number(cargos):
    """Dictionary comprehension: номер → стоимость."""
    return {cargo.number: cargo.cost for cargo in cargos}


def unique_statuses(cargos):
    return {cargo.status for cargo in cargos}


def heavy_numbers(cargos, limit):
    """List comprehension."""
    return [cargo.number for cargo in cargos if cargo.mass >= limit]
```

6. Примеры работы программы

На снимках показаны вывод коллекции, фильтр и map, сортировка и reduce, генератор, проверка any и собственный итератор.

```
C:\Users\Shmurdyakich\Desktop\pythonALL\python5> python main.py

Загружен учебный набор грузов: 6
Классы: ExpressCargo, StandardCargo

===== Обработка грузов =====
1. Показать все объекты
2. Отфильтровать доставленные
3. Получить номера
4. Отсортировать по массе
5. Общая стоимость доставки
6. Запустить генератор по статусу
7. Проверить наличие тяжёлых грузов
8. Собственный итератор
9. Сравнение списка и генератора
0. Выход

Выберите действие: 1

Все грузы:
 - G-101 [обычный] Озон → Иванов, 12.5 кг, 900 руб., доставлен
 - G-102 [срочный] М.Видео → Петрова, 4.2 кг, 1500 руб., в пути, срок 6 ч
 - G-103 [обычный] Склад Юг → Сидоров, 40.0 кг, 2400 руб., принят
 - G-104 [срочный] Озон → Кузнецова, 2.1 кг, 1100 руб., доставлен, срок 4 ч
 - G-105 [обычный] Ламода → Орлов, 8.0 кг, 700 руб., отменён
 - G-106 [срочный] Склад Юг → Иванов, 25.0 кг, 3200 руб., доставлен, срок 12 ч

===== Обработка грузов =====
1. Показать все объекты
2. Отфильтровать доставленные
3. Получить номера
4. Отсортировать по массе
5. Общая стоимость доставки
6. Запустить генератор по статусу
7. Проверить наличие тяжёлых грузов
8. Собственный итератор
9. Сравнение списка и генератора
0. Выход

Выберите действие: 2

Доставленные грузы (filter + lambda):
 - G-101 [обычный] Озон → Иванов, 12.5 кг, 900 руб., доставлен
 - G-104 [срочный] Озон → Кузнецова, 2.1 кг, 1100 руб., доставлен, срок 4 ч
 - G-106 [срочный] Склад Юг → Иванов, 25.0 кг, 3200 руб., доставлен, срок 12 ч

===== Обработка грузов =====
1. Показать все объекты
2. Отфильтровать доставленные
3. Получить номера
4. Отсортировать по массе
5. Общая стоимость доставки
6. Запустить генератор по статусу
7. Проверить наличие тяжёлых грузов
8. Собственный итератор
9. Сравнение списка и генератора
0. Выход

Выберите действие: 3
Номера (map): G-101, G-102, G-103, G-104, G-105, G-106
Та же выборка через функцию высшего порядка: G-101, G-102, G-103, G-104, G-105, G-106

===== Обработка грузов =====
1. Показать все объекты
```

Рисунок 1 – Список грузов, фильтр доставленных и получение номеров

```
C:\Users\Shmurdyakich\Desktop\pythonALL\python5> python main.py

Загружен учебный набор грузов: 6
Классы: ExpressCargo, StandardCargo

===== Обработка грузов =====
1. Показать все объекты
2. Отфильтровать доставленные
3. Получить номера
4. Отсортировать по массе
5. Общая стоимость доставки
6. Запустить генератор по статусу
7. Проверить наличие тяжёлых грузов
8. Собственный итератор
9. Сравнение списка и генератора
0. Выход

Выберите действие: 4

По возрастанию массы (sorted + key):
 - G-104 [срочный] Озон → Кузнецова, 2.1 кг, 1100 руб., доставлен, срок 4 ч
 - G-102 [срочный] М.Видео → Петрова, 4.2 кг, 1500 руб., в пути, срок 6 ч
 - G-105 [обычный] Ламода → Орлов, 8.0 кг, 700 руб., отменён
 - G-101 [обычный] Озон → Иванов, 12.5 кг, 900 руб., доставлен
 - G-106 [срочный] Склад Юг → Иванов, 25.0 кг, 3200 руб., доставлен, срок 12 ч
 - G-103 [обычный] Склад Юг → Сидоров, 40.0 кг, 2400 руб., принят

===== Обработка грузов =====
1. Показать все объекты
2. Отфильтровать доставленные
3. Получить номера
4. Отсортировать по массе
5. Общая стоимость доставки
6. Запустить генератор по статусу
7. Проверить наличие тяжёлых грузов
8. Собственный итератор
9. Сравнение списка и генератора
0. Выход

Выберите действие: 5
Общая стоимость (reduce): 9800 руб.
Стоимости по номерам: {'G-101': 900.0, 'G-102': 1500.0, 'G-103': 2400.0, 'G-104': 1100.0, 'G-105': 700.0, 'G-106':
3200.0}
Уникальные статусы: в пути, доставлен, отменён, принят
У всех грузов стоимость > 0: True

===== Обработка грузов =====
1. Показать все объекты
2. Отфильтровать доставленные
3. Получить номера
4. Отсортировать по массе
5. Общая стоимость доставки
6. Запустить генератор по статусу
7. Проверить наличие тяжёлых грузов
8. Собственный итератор
9. Сравнение списка и генератора
0. Выход

Выберите действие: 0
Работа программы завершена.
```

Рисунок 2 – Сортировка по массе и общая стоимость через reduce

```
C:\Users\Shmurdyakich\Desktop\pythonALL\python5> python main.py

Загружен учебный набор грузов: 6
Классы: ExpressCargo, StandardCargo

===== Обработка грузов =====
1. Показать все объекты
2. Отфильтровать доставленные
3. Получить номера
4. Отсортировать по массе
5. Общая стоимость доставки
6. Запустить генератор по статусу
7. Проверить наличие тяжёлых грузов
8. Собственный итератор
9. Сравнение списка и генератора
0. Выход

Выберите действие: 6
Статус (принят / в пути / доставлен / отменён): доставлен
Генератор выдаёт грузы со статусом «доставлен» по одному:
  yield → G-101 [обычный] Озон → Иванов, 12.5 кг, 900 руб., доставлен
  yield → G-104 [срочный] Озон → Кузнецова, 2.1 кг, 1100 руб., доставлен, срок 4 ч
  yield → G-106 [срочный] Склад Юг → Иванов, 25.0 кг, 3200 руб., доставлен, срок 12 ч

===== Обработка грузов =====
1. Показать все объекты
2. Отфильтровать доставленные
3. Получить номера
4. Отсортировать по массе
5. Общая стоимость доставки
6. Запустить генератор по статусу
7. Проверить наличие тяжёлых грузов
8. Собственный итератор
9. Сравнение списка и генератора
0. Выход

Выберите действие: 7
Порог массы, кг: 20
any(масса > 20.0): True
Номера не легче порога: ['G-103', 'G-106']

Дополнительно: стоимость меньше 1000 (функция высшего порядка):
 - G-101 [обычный] Озон → Иванов, 12.5 кг, 900 руб., доставлен
 - G-105 [обычный] Ламода → Орлов, 8.0 кг, 700 руб., отменён

===== Обработка грузов =====
1. Показать все объекты
2. Отфильтровать доставленные
3. Получить номера
4. Отсортировать по массе
5. Общая стоимость доставки
6. Запустить генератор по статусу
7. Проверить наличие тяжёлых грузов
8. Собственный итератор
9. Сравнение списка и генератора
0. Выход

Выберите действие: 0
Работа программы завершена.
```

Рисунок 3 – Генератор по статусу и проверка тяжёлых грузов

```
C:\Users\Shmurdyakich\Desktop\pythonALL\python5> python main.py

Загружен учебный набор грузов: 6
Классы: ExpressCargo, StandardCargo

===== Обработка грузов =====
1. Показать все объекты
2. Отфильтровать доставленные
3. Получить номера
4. Отсортировать по массе
5. Общая стоимость доставки
6. Запустить генератор по статусу
7. Проверить наличие тяжёлых грузов
8. Собственный итератор
9. Сравнение списка и генератора
0. Выход

Выберите действие: 8
Минимальная масса для итератора, кг: 10
Обход собственным итератором:
  next → G-101 [обычный] Озон → Иванов, 12.5 кг, 900 руб., доставлен
  next → G-103 [обычный] Склад Юг → Сидоров, 40.0 кг, 2400 руб., принят
  next → G-106 [срочный] Склад Юг → Иванов, 25.0 кг, 3200 руб., доставлен, срок 12 ч
StopIteration: элементы закончились

===== Обработка грузов =====
1. Показать все объекты
2. Отфильтровать доставленные
3. Получить номера
4. Отсортировать по массе
5. Общая стоимость доставки
6. Запустить генератор по статусу
7. Проверить наличие тяжёлых грузов
8. Собственный итератор
9. Сравнение списка и генератора
0. Выход

Выберите действие: 9

----- Список и генератор -----
Список номеров доставленных построен сразу: ['G-101', 'G-104', 'G-106']
type(list) = list
Генератор создан, элементы ещё не вычислены. type = generator
Первый next(): G-101
Второй next(): G-104
Остаток генератора: ['G-106']

===== Обработка грузов =====
1. Показать все объекты
2. Отфильтровать доставленные
3. Получить номера
4. Отсортировать по массе
5. Общая стоимость доставки
6. Запустить генератор по статусу
7. Проверить наличие тяжёлых грузов
8. Собственный итератор
9. Сравнение списка и генератора
0. Выход

Выберите действие: 0
Работа программы завершена.
```

Рисунок 4 – Собственный итератор и сравнение списка с генератором

7. Ответы на контрольные вопросы

1. Что такое функция высшего порядка?

Это функция, которая принимает другую функцию или возвращает её. В работе так устроены apply_to_each и select_where.

2. Зачем нужна lambda?

Она задаёт короткую функцию прямо в вызове map, filter или sorted, когда отдельное имя не нужно.

3. Что возвращает map?

Итератор, а не готовый список. Список получается через list(map(...)).

4. Чем filter отличается от обычного цикла?

filter оставляет элементы по предикату и тоже возвращает итератор. Смысл тот же, что у цикла с условием, но запись короче.

5. Когда уместен reduce?

Когда из многих элементов нужно одно значение, например сумма стоимостей. Для простой суммы часто понятнее sum().

6. Как sorted использует key?

key — функция, результат которой становится ключом сравнения. Здесь это lambda cargo: cargo.mass.

7. Чем any отличается от all?

any истинен, если условие выполнено хотя бы для одного элемента. all истинен, только если оно выполнено для всех.

8. Чем генератор отличается от списка?

Список хранит все элементы сразу. Генератор вычисляет очередное значение при запросе и экономит память.

9. Что делает yield?

Возвращает значение и запоминает место в функции. Следующий запрос продолжает выполнение после yield.

10. Что такое StopIteration?

Исключение, которым итератор сообщает, что элементы закончились. Цикл for перехватывает его сам.

8. Вывод

В лабораторной работе №5 коллекция грузов варианта 20 обработана функциональными средствами Python. Реализованы фильтр доставленных, получение номеров, сортировка по массе, общая стоимость, проверка тяжёлых грузов, генератор по статусу и собственный итератор. На сравнении списка и генератора видно различие между немедленным и ленивым вычислением.